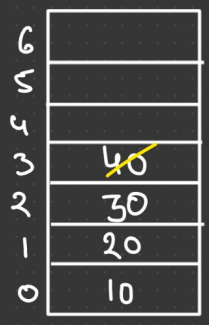
Stack

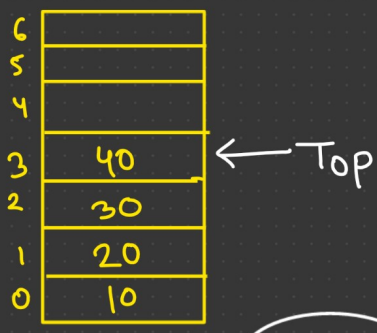
It is a data structure, which follows LIFO (Last in first out) property.



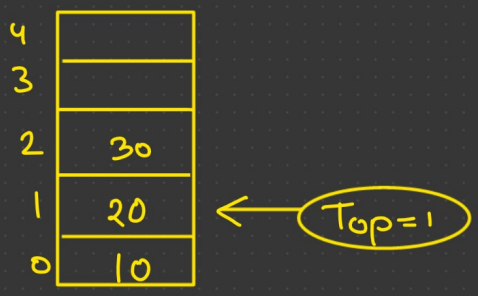



```
class Stack
{
    int top;
    int a[100];

  public :
      Stack()
      {
        top = -1;
      }
      Void push(int x)
      {
         if ( top == 99)
         {
            cout<<"Stack overflow";
            return;
         }
         else {
                top++;
                a[top] = x;
      }     }
```



Top = -1 → stack empty.



```
void pop()
{
    if(top== -1)
    {
     cout<<" underflow";
    }
    else
    {
        top--;
    }
}
```

```
int pop()
{
  if(top==-1)
   {
    cout<<"underflow";
    return -1;
   }
  else{ int x = a[top];
        top--;
        return x;
      }
}
```

```
int peek()
{
   if(top==-1)
   { cout<<"underflow";
     return -1;
    }
   else
      return a[top];
}
```

```
int isEmpty()
 {
    if(top==-1)
       return 1;
     else
       return 0;
  }

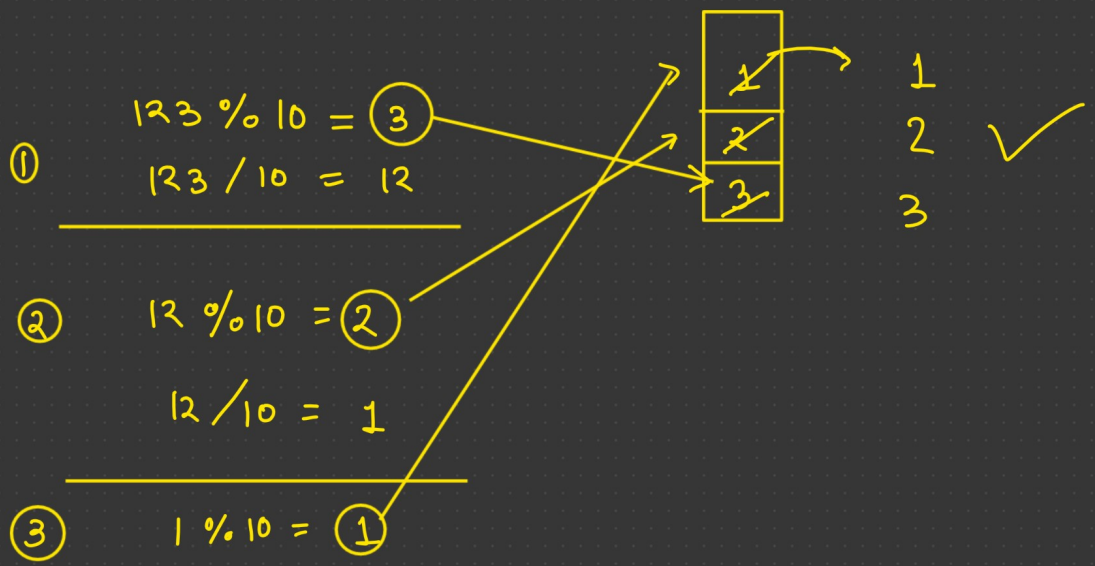
};
```

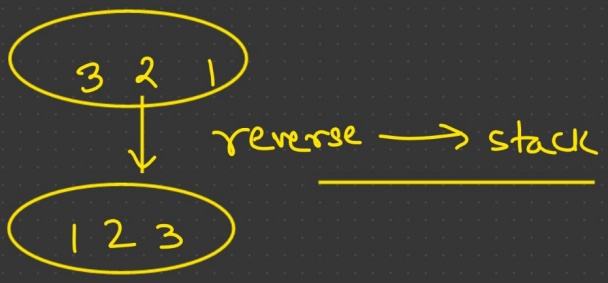
int x = 123;

1
2
3

X = 14632

1
4
6
3
2

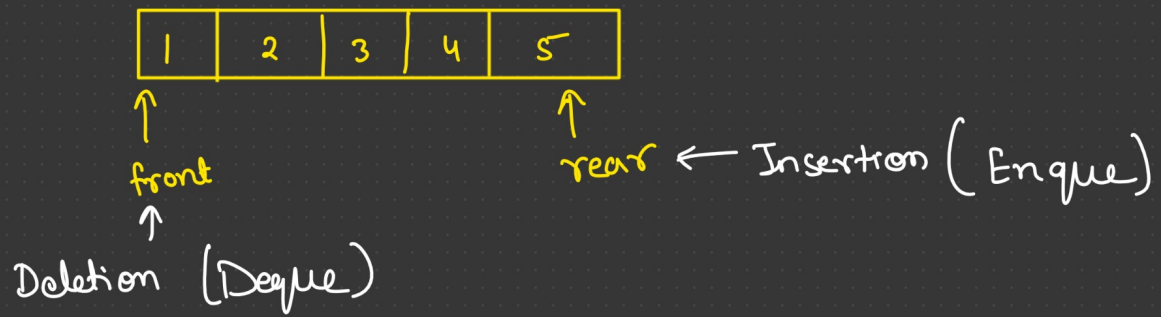





```
int main()
{  int x = 123;
   Stack s;
   while ( x != 0)
   {
      s.push( x %10);
      x = x / 10;
   }
      while( ! s.isEmpty())
   {
      Cout<< s.peek();
   }        s.POP();
   return 0;
}
```

Queue

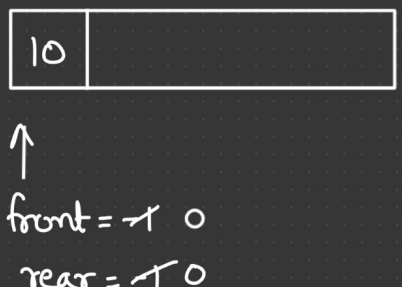
It is a Data structure, which follows FIFO (First in first out) property.



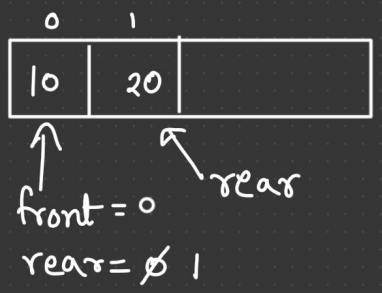
```
Class Queue
{
    int front, rear;
    int a[100];

  public:
        Queue()
        {
            front = rear = -1;
        }

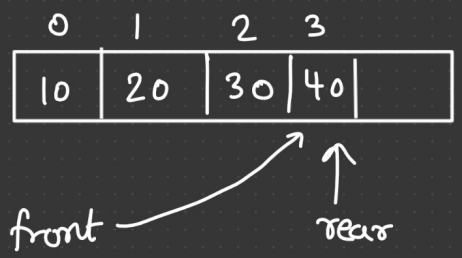
        void Enque(int x)
        {
            if (rear == 99)
                cout<<" Queue Overflow";
            else
            {
                if (front == -1 && rear == -1)
                {
                    front++; rear++;
                    a[rear] = x;
                }
```



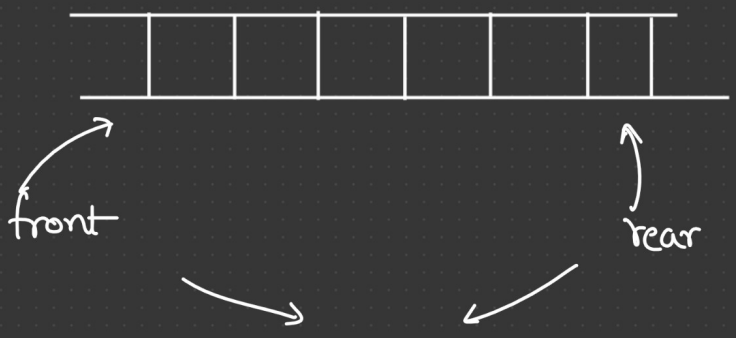
```
        else{
            rear ++;
            a[rear] = x;
        }
    }
}
```




```
void Dequel()
{
    if (front == -1)
    cout<<" Underflow";
    else
    {
        if (front == rear)
        {
            front = rear = -1;
        }
        else
            front ++;
    }
}
```

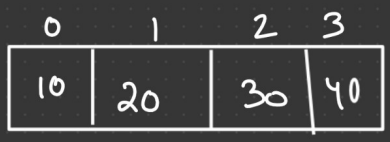


Double ended Queue



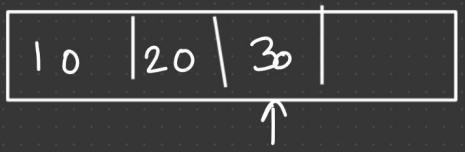
Both side insertion & deletion is allowed.

① push_front :-



front = ~~0~~ -1
rear = 3

② push_back :-



front = ~~0~~ 0
rear = ~~0~~ 1

3 pop front :-

